Day 2: Request Pipleline & Middleware

1. Request Pipeline

In ASP.NET Core, every HTTP request passes through a **middleware pipeline before reaching controllers**.

```
Client
↓
Middleware Pipeline
↓
Controller
↓
Application Layer
↓
Infrastructure Layer
↓
Database
```

Middleware can:

- inspect request
- modify request/response
- stop request
- pass request forward

2. Middleware Execution Model

Middleware executes in order on request and reverse order on response.

Request Flow	**Response Flow**
Client	Client
↓	↑
Middleware 1	Middleware 1
↓	↑
Middleware 2	Middleware 2
↓	↑
Middleware 3	Middleware 3
↓	↑
Controller	Controller

3. Middleware Location

Pipeline configuration happens in Program.cs, Middleware **order is critical**.

```
app.UseMiddleware<ExceptionHandlingMiddleware>();
app.UseMiddleware<RequestLoggingMiddleware>();
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

4. Middleware Structure

A middleware class typically contains-

- RequestDelegate _next

- HttpContext context
- InvokeAsync()

```
public class LoggingMiddleware
{
    private readonly RequestDelegate _next;
    public LoggingMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        // pre-processing
        await _next(context);
        // post-processing
    }
}
```

_next(context) forwards request to the **next middleware in the pipeline**.

5. Built-in Middleware Examples

ASP.NET Core provides built-in middleware:

- UseRouting()
- UseAuthentication()
- UseAuthorization()
- UseCors()
- UseHttpsRedirection()
- UseStaticFiles()

6. Custom Middleware Implemented

1. ExceptionHandlingMiddleware

Purpose:

- catch unhandled exceptions
- prevent server crash
- return structured JSON error

Example response:

```
{
  "error": "An unexpected error occurred.",
  "message": "Test exception",
  "traceId": "abc123"
}
```

2. RequestLoggingMiddleware

It logs

- HTTP Method
- Request Path
- Status Code
- Execution Time

Example log:

```
Incoming Request: POST /api/auth/login
Completed Request: POST /api/auth/login → 200 in 120ms
```

7. Middleware Order used

```
ExceptionHandlingMiddleware
↓
RequestLoggingMiddleware
↓
HttpsRedirection
```

```
↓
Authentication
↓
Authorization
↓
Controllers
```

Exception middleware must be **first** to catch all errors.

8. Testing Middleware

Test endpoint: GET /api/test/error

Controller intentionally throws:

```
throw new Exception("Test exception for middleware");
```

Middleware catches exception and returns structured JSON response.

9. Smart AI Dashboard Architecture Context

Middleware sits **before the API layer**.

```
HTTP Request
↓
Middleware
↓
Controller (API)
↓
Application Layer
↓
Infrastructure Layer
↓
Database
```

Day 2 – Implementation

1. What You Actually Built

You implemented a **custom middleware pipeline with production-grade concerns**:

Your pipeline now:

```
Request
 ↓
ExceptionHandlingMiddleware  (global safety net)
 ↓
RequestLoggingMiddleware     (observability)
 ↓
HTTPS Redirection
 ↓
Authentication (JWT)
 ↓
Authorization (Roles)
 ↓
Controller
```

2. ExceptionHandlingMiddleware (Critical System Layer)

File: ExceptionHandlingMiddleware.cs

```
try
{
   await _next(context);
}
catch (Exception ex)
{
   _logger.LogError(ex, "Unhandled exception occurred.");
   await HandleExceptionAsync(context, ex);
}
```

What's happening internally

- _next(context) → passes request to next middleware/controller
- If anything throws → caught here (GLOBAL CATCH)

Why this is powerful

Without this:

- App crashes OR
- Raw stack trace leaks (security risk)

With this:

- Controlled JSON response
- Centralized logging
- Production-safe system

Your response format

```
{
 "error": "An unexpected error occurred.",
 "message": "...",
 "traceId": "..."
```

}

Architectural Insight

You implemented **Cross-cutting concern handled at pipeline level (not controller level),** this is **Clean Architecture aligned**.

3. RequestLoggingMiddleware (Observability Layer)

File: RequestLoggingMiddleware.cs

```
var stopwatch = Stopwatch.StartNew();
_logger.LogInformation("Incoming Request: {Method} {Path}", ...);
await _next(context);
_logger.LogInformation("Completed Request...", statusCode, time);
```

What's happening internally

- Before _next: Logs incoming request
- After _next: Logs response + execution time

Why this is VERY important

- This is how companies:
- Debug slow APIs
- Monitor traffic
- Track failures
- Build dashboards (future)

Architectural Insight

- You just implemented Observability (one of the pillars of distributed systems)
- This is what tools like Datadog, ELK Stack, Azure Monitor …do at scale.

4. Middleware Order

Program.cs

```
app.UseMiddleware<ExceptionHandlingMiddleware>();
app.UseMiddleware<RequestLoggingMiddleware>();
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

Why this order matters

- Exception FIRST Catches everything below it
- Logging BEFORE business logic Logs ALL requests (even failed ones)

Authentication before Authorization

app.UseAuthentication(); → sets User
app.UseAuthorization(); → checks User

- If reversed → ❌ always fails

5. TestController

File: TestController.cs

```
[HttpGet("error")]
public IActionResult ThrowError()
{
    throw new Exception("Test exception for middleware");
}
```

Why this is smart

You created a **controlled failure endpoint**.

Used to verify:

- Exception middleware works
- Logging middleware captures failure
- Response format is correct

What Happens When You Hit /api/test/error

Let's trace FULL FLOW:

Step-by-step execution

1. Request enters Kestrel
2. ExceptionMiddleware starts
3. LoggingMiddleware logs "Incoming"
4. Controller throws exception
5. Exception bubbles up
6. ExceptionMiddleware catches it
7. JSON error returned
8. LoggingMiddleware logs completion

6. Dependency Injection

Infrastructure/DependencyInjection.cs

```
services.AddScoped<IAIProvider, OpenAIProvider>();
services.AddScoped<IAIService, AiService>();
services.AddScoped<IJwtTokenService, JwtTokenService>();
```

What this means

At runtime:

IAIService → AiService

IAIProvider → OpenAIProvider

Why this matters

- Loose coupling
- Easy to replace OpenAI later
- Testable system

7. JWT Authentication Integration

Program.cs

You configured: builder.Services.AddAuthentication(...)

What happens in pipeline

When request comes:

1. Middleware reads token
2. Validates signature

3. Extracts claims
4. Sets HttpContext.User

Then Authorization uses it:

[Authorize(Roles = Roles.User)]